

INCEPTION

Correction, Testing & Defense Roadmap

Prepared around the current ChimexXy/INCEPTION_SE project on branch `intra`

What this document is for

A practical study sheet to get the project ready for correction and defense. It combines the correction-sheet flow with the actual architecture and commands found in the current repository.

Important repository note

The branch `intra` is the branch used here. Before correction, keep documentation synchronized with the actual Makefile commands and verify the final repository from a clean clone.

How to use this guide

Do not try to memorize the project line by line. First understand the architecture, then reproduce every test from the terminal, then practice explaining each design choice in your own words.

- Stage 1: build the project from a clean state and prove that it works.
- Stage 2: verify every mandatory requirement independently.
- Stage 3: verify persistence and configuration changes.
- Stage 4: run and demonstrate every bonus service.
- Stage 5: practice the defense questions until you can explain the project without reading notes.

Contents

1. Correction mindset and project map
2. Repository and file-by-file map
3. Complete mandatory test roadmap
4. Mandatory defense topics
5. Bonus test roadmap
6. Configuration-modification and persistence tests
7. Documentation readiness
8. Final correction checklist
9. Defense questions and model answers
10. Commands to know by heart
11. What to fix or verify before the final correction

1. Correction mindset and project map

The correction should be approached as a sequence of independent proofs. For every requirement, ask: what is the expected behavior, what command proves it, and why did I implement it this way?

1.1 Core architecture

HOST / VM

```

Browser ---- HTTPS :443 ----> NGINX
                                |
                                | FastCGI :9000
                                v
                        WORDPRESS + PHP-FPM
                                |
                                | TCP :3306
                                v
                        MARIADB
  
```

Docker network: inception (bridge)

Named volumes:

```

inception_wordpress_files -> /home/mozahnou/data/wordpress
inception_mariadb_data    -> /home/mozahnou/data/mariadb
  
```

One sentence to memorize

NGINX is the public HTTPS entry point, WordPress runs PHP through PHP-FPM, MariaDB stores the application data, Docker networking connects the services internally, and named volumes keep persistent data outside the containers.

1.2 Mandatory vs bonus

Part	Services	Main idea
Mandatory	nginx, wordpress, mariadb	Three separate services, custom-built images, HTTPS entry point, database and persistent volumes.

Bonus	redis, ftp, adminer, static-site, status	Optional extra services integrated with the mandatory stack and exposed through internal networking / NGINX as designed.
-------	--	--

2. Repository and file-by-file map

You should be able to open any important file during the defense and immediately explain why it exists.

Path	Purpose
Makefile	Entry point for setup, mandatory build/start, bonus build/start, cleaning and rebuild.
srcs/docker-compose.yml	Mandatory three-service infrastructure.
srcs/docker-compose.bonus.yml	Bonus overlay that adds the bonus services and bonus-only behavior.
srcs/.env	Non-secret configuration such as domain, DB names/users, WordPress settings and bonus service coordinates.
secrets/*.txt	Passwords used as Docker secrets; generated locally and ignored by Git.
requirements/nginx	NGINX Dockerfile, vhost templates and entrypoint.
requirements/wordpress	WordPress/PHP-FPM Dockerfile, PHP-FPM pool configuration and entrypoint.
requirements/mariadb	MariaDB Dockerfile, database configuration and entrypoint.
requirements/bonus/*	Independent Dockerfiles/configuration for bonus services.
README.md	Project description, instructions, resources and AI-use documentation.
USER_DOC.md	How to run and use the infrastructure.
DEV_DOC.md	Developer-oriented architecture, setup and modification notes.

2.1 Current `intra` configuration values

Variable	Current purpose
LOGIN	Host/login identity used by the project.
DOMAIN_NAME	Main `.42.fr` domain used by NGINX and WordPress.
DATA_PATH	Host location backing the two named volumes.
MYSQL_HOST	Database service name used by WordPress.
MYSQL_DATABASE	WordPress database name.
MYSQL_USER	Non-root DB account used by WordPress.
WP_URL / WP_TITLE	WordPress public URL and site title.
WP_ADMIN_USER / WP_ADMIN_EMAIL	WordPress administrator identity.
WP_USER / WP_USER_EMAIL	Second WordPress user.
REDIS_HOST / REDIS_PORT	Bonus Redis connection details.
FTP_USER / FTP_PASV_MIN / FTP_PASV_MAX	Bonus FTP settings.

3. Complete mandatory test roadmap

Run these tests in order. The sequence starts from the repository and build, then moves through Docker, networking, TLS, WordPress, MariaDB, volumes and persistence.

3.1 Start from a clean state

```
git checkout intra
git pull origin intra
git status

# Destructive: removes project data
make fclean
make
```

Clean-state rule

Use `make fclean` only when you are ready to erase the WordPress and MariaDB data. For normal iteration, rebuild/recreate only the affected service.

3.2 Check the containers

```
docker ps
docker compose -f srcs/docker-compose.yml ps
```

Expected mandatory services: nginx, wordpress and mariadb. MariaDB should report healthy once its healthcheck passes.

3.3 Validate the Compose model

```
docker compose -f srcs/docker-compose.yml config
docker compose -f srcs/docker-compose.yml config --services
```

The mandatory compose file should declare exactly the three mandatory services. The bonus overlay should be separate.

3.4 Check the Docker network

```
docker network ls
docker network inspect inception

docker exec wordpress getent hosts mariadb
docker exec nginx getent hosts wordpress
```

Explain that Docker provides service-name resolution on the user-defined bridge network. `wordpress` resolves `mariadb`; NGINX resolves `wordpress`.

3.5 Check the volumes

```
docker volume ls
docker volume inspect inception_wordpress_files
docker volume inspect inception_mariadb_data

ls -la /home/mozahnou/data/wordpress
ls -la /home/mozahnou/data/mariadb
```

Be ready to explain the difference between a named Docker volume and a bind mount, and why this project uses named volumes backed by the required host paths.

3.6 Check logs and startup behavior

```
docker logs nginx
docker logs wordpress
docker logs mariadb
```

Know the expected startup sequence: MariaDB initializes only on a cold data directory; WordPress waits for MariaDB and installs only when `wp-config.php` is missing; NGINX renders configuration, creates TLS material if needed, runs `nginx -t`, then starts NGINX in the foreground.

3.7 WordPress HTTP/HTTPS tests

```
curl -kI https://mozahnou.42.fr
curl -kI https://mozahnou.42.fr/wp-login.php

# Port 80 should not be the web entry point
curl -I --connect-timeout 3 http://mozahnou.42.fr
```

The certificate is self-signed, so `curl -k` is used for local verification. In a browser, expect a certificate warning.

3.8 Test WordPress through WP-CLI

```
docker exec wordpress wp --allow-root --path=/var/www/html user list

docker exec wordpress wp --allow-root --path=/var/www/html option get siteurl

docker exec wordpress wp --allow-root --path=/var/www/html db check

# Optional: check plugins
docker exec wordpress wp --allow-root --path=/var/www/html plugin list
```

What you must understand

NGINX does not execute PHP. It forwards PHP requests with FastCGI to `wordpress:9000`, where PHP-FPM executes the WordPress PHP application.

3.9 WordPress admin test

Open the main site and log into `/wp-admin`. Create or edit a piece of content. Then refresh the page and verify the change is really stored.

```
https://mozahnou.42.fr/wp-admin
```

3.10 MariaDB tests

```
docker exec mariadb mariadb-admin ping \
--socket=/run/mysqld/mysqld.sock \
-u root -p"${cat secrets/db_root_password.txt}"

# Interactive DB shell
docker exec -it mariadb sh -c \
'mariadb -u root -p"${cat /run/secrets/db_root_password}" wordpress'
```

Inside MariaDB you should be able to run `SHOW DATABASES;`, inspect the WordPress database, and explain why root access is deliberately handled from inside the database container rather than published on the host.

3.11 Check host exposure

```
docker ps
ss -lnt

# Inspect individual containers
docker inspect nginx
docker inspect wordpress
docker inspect mariadb
```

Mandatory expectation: NGINX exposes 443; WordPress and MariaDB do not publish their application ports to the host.

3.12 TLS tests

```
openssl s_client -connect mozahnou.42.fr:443 -tls1_2  
openssl s_client -connect mozahnou.42.fr:443 -tls1_3
```

Know what the command proves: the TLS handshake, negotiated protocol and certificate information. Your NGINX configuration explicitly allows TLS 1.2 and TLS 1.3.

4. Mandatory defense topics

Docker image vs container

An image is the template used to create containers; a container is a running instance of an image.

Dockerfile vs Compose

A Dockerfile builds one service image. Compose defines how multiple services, networks, volumes, secrets and dependencies work together.

Docker vs VM

A VM includes a guest OS/kernel. Containers share the host kernel and isolate processes with Linux primitives such as namespaces and cgroups.

Why separate containers?

Each major service has its own responsibility: web entry point, application runtime, database. This matches the project architecture and keeps the services independently manageable.

Bridge network

The user-defined ``inception`` bridge gives containers network isolation plus service-name DNS.

No host network

Host networking would remove the intended network isolation and bypass the Compose bridge network model.

No ``links``

Compose service discovery is already provided by the user-defined network and DNS; ``links`` is unnecessary.

PID 1

The real daemon is intended to become PID 1 so signals from Docker reach the service directly.

Why ``exec``?

``exec`` replaces the shell with the daemon instead of leaving a shell wrapper as PID 1.

Why foreground mode?

NGINX and PHP-FPM are configured to stay in the foreground so the real service remains the container process.

Why no ``tail -f`` / ``sleep infinity``?

Those are artificial keep-alive tricks. The container should stay alive because its real service is running.

NGINX + PHP-FPM

NGINX handles HTTPS and HTTP routing; PHP-FPM handles PHP execution.

WordPress + MariaDB

WordPress is the application; MariaDB stores its persistent relational data.

Named volumes

Containers can be recreated while persistent data remains in volumes.

Secrets vs env

Normal configuration can be in ``.env``; passwords are kept as Docker secrets mounted under ``.run/secrets/``.

Idempotent initialization

MariaDB skips initialization when its data directory is already initialized; WordPress skips installation when ``.wp-config.php`` already exists.

4.1 NGINX request flow

```

Browser
|
| HTTPS :443
v
NGINX
|
| PHP request -> FastCGI
v
wordpress:9000
|
v
PHP-FPM
|
v
WordPress
|
v
MariaDB

```

4.2 Why MariaDB data is initialized at runtime

The MariaDB image installs the database server package during image creation, then clears the package-created database contents. On first volume use, the entrypoint performs database initialization explicitly. This avoids accidentally treating package-created files as an already initialized project database.

5. Bonus test roadmap

Start with the full bonus stack:

```
make down
make bonus

docker compose \
  -f srcs/docker-compose.yml \
  -f srcs/docker-compose.bonus.yml \
  config --services

docker ps
```

Bonus service	What to prove	Example test
Redis	WordPress object cache is active and Redis answers	`docker exec redis redis-cli PING` and `wp redis status`
FTP	FTP login, passive mode, and shared WordPress volume	Login + upload a test file + fetch it through HTTPS
Adminer	Web DB UI works behind NGINX	Open `https://adminer.mozahnou.42.fr`
Static site	HTML/CSS/JS site works and contains no PHP	Open `https://static.mozahnou.42.fr`
Status	Custom service reports stack health	Open `https://status.mozahnou.42.fr` and `/health`

5.1 Redis

```
docker exec redis redis-cli PING

docker exec wordpress wp --allow-root --path=/var/www/html redis status
```

Understand `maxmemory` and `allkeys-lru`: Redis is treated as a bounded cache. The cache can be discarded without losing the source-of-truth data stored in MariaDB.

5.2 FTP

```
cat secrets/ftp_password.txt

curl --ftp-pasv \
  -u "ftp_user:$(cat secrets/ftp_password.txt)" \
  ftp://127.0.0.1:21/
```

Then upload a small file in passive mode. Verify the file lands in the shared WordPress volume and is served through the public HTTPS site.

5.3 Adminer

```
curl -kI https://adminer.mozahnou.42.fr
```

Be able to explain the path: browser -> NGINX HTTPS -> Adminer on the internal Docker network -> MariaDB.

5.4 Static site

```
curl -kI https://static.mozahnou.42.fr
```

The static site is a separate service. The public TLS termination still happens in the main NGINX container.

5.5 Status service

```
curl -k https://status.mozahnou.42.fr  
curl -k https://status.mozahnou.42.fr/health
```

Understand the distinction between a container being in `running` state and the application actually answering requests.

6. Configuration-modification and persistence tests

6.1 Persistence test

1. Create a WordPress post.
2. Remember the title.
3. make down
4. make
5. Open the site again.
6. Confirm the post still exists.
7. Reboot the VM.
8. Start Docker.
9. make
10. Confirm the post still exists.

This is the practical proof that application state belongs in persistent volumes rather than in ephemeral container layers.

6.2 Configuration modification test

Practice changing a small, reversible service configuration and rebuilding only the affected service. A good example is changing the internal PHP-FPM listen port, then changing NGINX `fastcgi_pass` to match. The goal is not to make the stack different permanently; the goal is to prove you understand the dependency between two services and can rebuild safely.

Example:

```
WordPress PHP-FPM: listen = 0.0.0.0:9001
NGINX: fastcgi_pass wordpress:9001;
```

Rebuild/recreate the relevant services, verify the site, then revert the change.

Defense point

The evaluator may ask you to modify configuration. You should know exactly which file controls the behavior, which other service depends on it, and which rebuild/recreate command applies the change.

7. Documentation readiness

Document	What it should contain	Final check
README.md	Description, instructions, resources, AI-use documentation.	Commands and filenames must match the actual `intra` branch.
USER_DOC.md	Starting/stopping, access, credentials, bonus usage, basic troubleshooting.	No commands that do not exist in the final Makefile.
DEV_DOC.md	Prerequisites, repository map, <code>.env</code> , secrets, Compose, Dockerfiles, entrypoints, volumes, network, rebuild workflow.	Explain the implementation rather than only listing commands.

Current repository warning

The earlier project documentation still contains references to convenience targets such as ``make test``, ``make ps``, ``make logs``, ``make stop``, ``make start``, ``make hosts``, and ``make help``, while the simplified Makefile no longer defines all of them. Before correction, either restore those targets or update the documentation so there are no dead commands.

8. Final correction checklist

- [] Make works from a clean state.
- [] Make bonus works from a clean state.
- [] Mandatory compose contains exactly nginx, wordpress and mariadb.
- [] Every mandatory service has its own Dockerfile.
- [] No `latest` image tag is used.
- [] No host networking is used.
- [] No deprecated `links` configuration is used.
- [] No artificial keep-alive process is used.
- [] Real service is PID 1 in each mandatory container.
- [] NGINX is the public HTTPS entry point on port 443.
- [] Port 80 is not the public web entry point.
- [] TLS 1.2 and TLS 1.3 work.
- [] WordPress loads successfully.
- [] WordPress login works.
- [] Two WordPress users exist and the administrator login is valid for the subject restrictions.
- [] PHP is executed through PHP-FPM.
- [] MariaDB is reachable from WordPress.
- [] MariaDB is not published to the host.
- [] Both named volumes exist and map to the required host data locations.
- [] WordPress data survives `make down` and restart.
- [] Database data survives `make down` and restart.
- [] Credentials are handled as secrets rather than normal `.env` values.
- [] WordPress configuration is not downloadable as a web file.
- [] Admin dashboard content can be created/edited.
- [] Reboot persistence has been tested.
- [] Bonus Redis works.
- [] Bonus FTP works.
- [] Bonus Adminer works.
- [] Bonus static site works.
- [] Bonus status service works.
- [] README matches the actual project.
- [] USER_DOC matches the actual Makefile.
- [] DEV_DOC matches the actual directory structure.

9. Defense questions and model answers

Q: What is Docker?

A: A platform for running applications in isolated containers built from images.

Q: What is an image?

A: A reusable filesystem/configuration template used to create containers.

Q: What is a container?

A: A running instance of an image.

Q: Why Docker Compose?

A: To define and manage a multi-container application as one declarative project.

Q: Why not use one big container?

A: The architecture separates NGINX, WordPress/PHP-FPM and MariaDB into independent services.

Q: Why a bridge network?

A: It gives internal network isolation and service-name DNS between containers.

Q: Why can WordPress use `mariadb` as a hostname?

A: Compose connects the services to the same user-defined network and provides service-name DNS.

Q: Why is MariaDB not published?

A: WordPress needs it internally, but the database should not be directly reachable from the host.

Q: Why does NGINX talk to port 9000?

A: PHP-FPM listens on 9000 and accepts FastCGI requests.

Q: What happens if PHP-FPM is down?

A: NGINX cannot complete PHP requests and the site will typically return a gateway error.

Q: Why do you use a named volume?

A: To keep persistent application/database data outside disposable containers.

Q: What is the difference between a volume and a bind mount?

A: A named volume is a Docker-managed volume object; a bind mount directly maps a host path into a container. This project uses named volumes backed by the required host directories.

Q: Why use secrets?

A: Passwords are sensitive credentials and are kept separate from ordinary non-secret configuration.

Q: Why not put passwords in `.env`?

A: The project deliberately separates configuration from credential material.

Q: Why use `exec` in entrypoints?

A: It replaces the shell so the actual daemon becomes PID 1 and receives signals directly.

Q: Why not `tail -f /dev/null`?

A: That would keep a container alive without running the real service.

Q: Why does MariaDB initialize only when the data directory is empty?

A: To make startup idempotent and avoid destroying an existing database.

Q: Why does WordPress check for `wp-config.php`?

A: It uses the presence of the config file as a first-installation marker so restarts do not reinstall the site.

Q: Why is the TLS certificate self-signed?

A: This is a local `.42.fr` development/correction setup; there is no public CA relationship for the local certificate.

Q: What is Redis doing?

A: It provides an object-cache layer so WordPress can reuse cached application data instead of rebuilding it every time.

Q: Why does FTP need passive ports?

A: FTP has a control connection and separate data connections; passive mode uses a configurable port range for data channels.

Q: Why does Adminer not publish its own host port?

A: It is reached through the main NGINX reverse proxy over the internal Docker network.

Q: What is your service of choice?

A: The project adds a custom Python status dashboard that probes the other services and exposes HTML plus JSON health information.

10. Commands to know by heart

```
git checkout intra
git pull origin intra
git status
```

```
make
make bonus
make down
make clean
make fclean
make re
```

```
docker ps
docker logs <container>
docker exec -it <container> sh
docker inspect <container>
docker network inspect inception
docker volume inspect inception_wordpress_files
docker volume inspect inception_mariadb_data
```

```
docker compose -f srcs/docker-compose.yml config
docker compose -f srcs/docker-compose.yml config --services
```

```
curl -kI https://mozahnou.42.fr
openssl s_client -connect mozahnou.42.fr:443 -tls1_2
openssl s_client -connect mozahnou.42.fr:443 -tls1_3
```

```
docker exec wordpress wp --allow-root --path=/var/www/html user list
docker exec wordpress wp --allow-root --path=/var/www/html db check
docker exec redis redis-cli PING
```

11. What to fix or verify before the final correction

- Make sure your local checkout is really on `intra` and you have pulled the latest commits.
- Run `make fclean && make` from a clean state before the correction.
- Run `make bonus` from a clean state and test all five bonus services.
- Synchronize README.md, USER_DOC.md and DEV_DOC.md with the final Makefile and filenames.
- Do not leave temporary test modifications in the repository.
- Verify that secrets are not tracked and that only non-secret configuration is committed.

- Practice the persistence test and the configuration-modification test before the defense.

Defense principle

For every feature, know three things: what file implements it, what command demonstrates it, and what technical reason justifies it.

Source basis

Primary source: the Inception correction sheet supplied in this conversation. Repository-specific implementation details in this guide were checked against the current `intra` branch of ChimexXy/INCEPTION_SE during preparation.